目录

Int4 原生计算 — RISC-V Vector 指令需求报告
执行摘要
1. 精度验证结果
2. 内核算术分析
3. CUDA Int4 指令设计参考
4. 提出的 RVV Int4 指令语义
5. 软件需求汇总
6. 与 RISC-V Matrix Extension (IME) 的关系
7. 参考文献
附录 A: 指令编码占位符
附录 B: 验证测试向量
附录 C: 术语表

Int4 原生计算 — RISC-V Vector 指令需求报告

日期: 2026-04-28 | 作者: int4-report（软件团队） | 目标受众: 硬件团队 平台: RISC-V RVV VLEN=512 (zvl512b) | 分析层级: 软件需求 — 不含硬件可行性分析

执行摘要

基于在两个真实负载（llama.cpp Q4_0 LLM 推理和 ONNX Runtime YOLO11n-INT8 目标检测）上的精度验证，RISC-V Vector 原生 int4 计算指令是可行且有充分理由的。软件团队请求硬件团队评估以下指令需求。

精度证据

负载	内核	热点占比	Int4 仅权重 CosSim	Int4 完整 CosSim	精度损失
llama.cpp Qwen2.5-0.5B Q4_0	GEMV（Q4_0×Q8_0）	53.93%	—	~0.98+（估计）	低 — 事实输出一致，生成连贯
ONNX RT YOLO11n-INT8	QGEMM（u8×u8→i32）	74.51%	0.989	~0.915（估计）	低—中等 — 仅权重可用；完整 int4 需谨慎

关键发现: 两个负载均能容忍 int4 量化，精度损失可接受。占主导地位的计算内核（LLM 推理中 GEMV 占 53.93%，目标检测中 QGEMM 占 74.51%）将直接受益于原生 int4 指令。

算术改进空间

内核	当前每 MAC 操作数	使用原生 Int4	预估 K 循环加速比
llama.cpp GEMV	vand + vsrl + vwmacc.vx（3 操作）	1 操作（vqmac.vx）	~2.5×
ONNX RT QGEMM	vle8 + vwmulu.vx + vwaddu.wv（3 操作）	vle8 + vqmac.vv（2 操作）	~3.4×

请求的指令（汇总）

1. `vqmac.vv.i4.i32` — 打包 int4 向量-向量四路乘累加（有符号×有符号→i32）
2. `vqmac.vx.i4.i32` — 打包 int4 向量-标量四路乘累加（有符号标量 × 打包向量→i32）
3. `vqmacu.vv.i4.i32` / `vqmacsu.vv.i4.i32` — 无符号和混合符号变体，用于非对称量化

这些指令遵循 CUDA `dp4a` 设计模式（打包子字节点积带累积），并适配 RVV 的向量长度无关（vector-length-agnostic）LMUL 基础设施。

1. 精度验证结果

1.1 llama.cpp — LLM 推理（Qwen2.5-0.5B Q4_0）

平台: Banana Pi K1（SpacemiT X60, VLEN=256） | 模型: Qwen2.5-0.5B-Instruct Q4_0 热点内核: `ggml_gemv_q4_0_16x1_q8_0`（23.50%）+ `ggml_gemv_q8_0_16x1_q8_0`（29.43%）= 合计 53.93% GEMV

量化方案: 对称 int4 [-7, 7]，逐行缩放，存入 int8 容器。现有 GEMV 内核无修改消费。

结果（5 个提示词，温度 0.0 和 0.7）：

评价标准	结果
事实准确性	✅ 与 int8 基线一致（法国首都、水的化学式、光速）
推理能力	✅ 识别出相同的公式，输出质量相近
连贯性	✅ 所有输出语法正确、切题
创意生成（T=0.7）	⚠✅ 观察到语言切换（英文→中文），但内容主题符合
灾难性失败	✅ 无 — 无乱码、空输出或崩溃

评估: 低精度损失。Int4 激活量化保留了足够的信息用于连贯的自回归文本生成。占主导地位的 GEMV 模式（标量激活 × 权重向量）自然地映射到所提出的 `vqmac.vx` 指令。

来源: `docs/report/int4-instruction/llama-int4-precision-2026-04-28.md`

1.2 ONNX Runtime YOLO11n — 目标检测

平台: QEMU VLEN=512 | 模型: YOLO11n-INT8 | 测试图像: COCO `bus.jpg` 热点内核: `MlasQgemmKernel`（占总运行时间 74.51%，Banana Pi K1 硬件 perf 确认）

量化方案: 非对称逐通道 uint4 [0, 15]，逐通道缩放因子 + 零点偏移。 INT4 仅权重（激活保持不变）：权重量化至 uint4，激活值不变，内核无修改消费。

INT4 仅权重结果:

指标	INT8 基线	INT4 仅权重	变化
余弦相似度（vs FP32）	0.999	0.989	−0.010
MSE（vs FP32）	5.75	78.54	+13.7×
Top-20 检测重叠	17/20（85%）	8/20（40%）	−45pp
高置信度检测 (>0.5)	47	10	−79%

逐层分析: INT4/INT8 MSE 比率 ≈ 256× 贯穿所有层次组，精确匹配量化噪声理论（`(255/15)^2 = 289`，逐通道适配后降低）。Stem 层（`model.0.conv`, shape 16×3×3×3）和检测头层最为敏感。

完整 INT4 估计（权重+激活）：预估余弦相似度 ~0.915（适中抵消因子 α=0.3）。保守估计: 0.744；乐观: 0.964。

评估: 低—中等精度损失（仅权重，CosSim=0.989，可直接使用）。中等损失（完整 INT4，CosSim≈0.915，需要精度权衡分析）。混合精度（int4 权重 + int8 激活）是 QGEMM 负载的最佳平衡

点。

来源: docs/report/int4-instruction/yolo-int4-precision-2026-04-28.md

1.3 跨负载对比

属性	llama.cpp（LLM）	ONNX RT YOLO（视觉）
任务类型	自回归文本生成	稠密空间预测
主导计算	GEMV（标量×向量）	GEMM（矩阵×矩阵）
权重格式	Q4_0（4-bit，每字节 2 个）	INT8（8-bit，每字节 1 个）
激活格式	Q8_0（逐行对称）	uint8（逐张量非对称）
Int4 容忍度	高（事实输出保持）	中等（置信度降低）
误差累积	中等（自回归链）	高（多层特征图）
模型大小	0.5B 参数	2.6M 参数（冗余更少）
推荐 int4 模式	完整 int4（权重+激活）	混合（权重 int4，激活 int8）

YOLO 更敏感的原因：稠密预测在每个空间位置放大误差；小模型（2.6M 参数）冗余少；逐张量激活量化对 4-bit 太粗糙；Stem 层（3 输入通道）最难量化。

2. 内核算术分析

2.1 llama.cpp GEMV/GEMM — ggml_gemm_q4_K_8x4_q8_K

文件: applications/llama.cpp/rvv-patches/gemm-q4_K-8x4-q8_K/rvv_gemm_q4_K_8x4.inl

当前计算模式（内循环）：

```
// 步骤 1: 加载打包的 Q4_K 权重（每字节 2 个权重，stride=4）
q4_packed = vlse8(weights + k*32 + i, stride=4); // vl=8，加载 8 字节

// 步骤 2: 将 nibble 解包为 int8（2 条指令）
q4_lo = vand(q4_packed, 0xF); // 提取低 nibble → int8
q4_hi = vsrl(q4_packed, 4); // 提取高 nibble → int8

// 步骤 3: 标量激活 × 扩展乘累加（每行 1 条指令）
acc_row = vwmacc.vx(acc_row, q8_activation_scalar, q4_lo);
acc_row = vwmacc.vx(acc_row, q8_activation_scalar, q4_hi);
// 结果: i16 部分和（8 个元素）
```

每次内循环迭代的算术密度（VLEN=512，vl=8）：

- 加载: 8 字节打包权重 = 16 个 int4 权重值
- 操作: 2 解包（vand + vsrl）+ 2×4 vwmacc.vx = 10 条向量操作 处理 16 权重 × 4 行 = 64 个 MAC
- 每个 MAC 仅有 0.16 条有用操作（其余为解包开销）

原生 int4 的实现形式:

```
// 使用 vqmac.vx: 标量 int8 激活 × 打包 int4 向量 → int32 累积
// vl=16: 16 字节 = 32 个打包 int4 权重，一条指令处理
v_packed_i4 = vle8(weights + k*32 + i); // vl=16，加载 16 字节
acc_row = vqmac.vx.i4.i32(acc_row, q8_scalar, v_packed_i4);
// 32 个 int4 权重 × 标量 int8 → 累积至 16 个 int32 输出（每输出 2 个权重）
```

预估改进: 2 条指令（vle8 + vqmac.vx）替代 10 条操作。解包开销完全消除。

2.2 ONNX Runtime QGEMM — MlasQgemmKernel

文件: applications/onnxrt/rvv-patches/qgemm-kernel-vl16/rvv_qgemm_kernel_vl16.inl

当前计算模式（K 内循环）：

```
// 每个 PackedK 组（4 个 K 元素 × 16 列）：
a0, a1, a2, a3 = A[0..3]; // 4 个标量激活值

// K 元素 0:
vb0 = vle8(B); // 加载 16 个 uint8 权重
vp0 = vwmulu.vx(vb0, a0); // u8×u8→u16 扩展乘法
vacc = vwaddu.wv(vacc, vp0); // u32 += u16 扩展累加

// K 元素 1-3 重复（相同模式）
// 总计: 4 vle8 + 4 vwmulu.vx + 4 vwaddu.wv = 12 条操作/PackedK 组
```

算术密度（每 PackedK 组）：

- 4 个标量激活 × 16 列 = 64 个 MAC
- 12 条向量操作 / 64 MACs = 每个操作 5.3 个 MAC
- 每个 MAC 需要 2 条操作（vwmulu.vx + vwaddu.wv）

原生 int4 的实现形式（仅 int4 权重）：

```
// 使用 vqmacu.vv: 打包 uint4 权重 × uint8 激活 → uint32 累积
// vl=16: 16 字节打包 int4 权重（32 个值）× 16 个 uint8 激活

v_packed_u4 = vle8(B); // vl=16, 加载 16 字节 = 32 个打包 uint4 权重

// 四路乘累加: 4 个打包 uint4 × 4 个打包 uint4 激活 → uint32
vacc = vqmacsu.vx.i4.i32(vacc, a0, v_packed_u4);
// 无符号 int4 × 有符号 int8 标量
```

预估改进:

- 内存: int4 权重加载字节数减少 2×
- 计算: 每 PackedK 元素 1 条指令替代 3 条（vwmulu + vwaddu → vqmac）
- K 循环: 当前 ~11 周期/K 元素，原生 int4 ~3 周期/K 元素 → ~3.4× 加速

2.3 瓶颈汇总

内核	瓶颈	Int4 影响
llama.cpp GEMV	解包 i4→i8（2 条操作）+ int8 MAC	消除解包；单条打包指令
ONNX RT QGEMM	两步扩展 MAC（vwmulu + vwaddu）	单步打包 MAC
两者	权重内存带宽	int4 权重节省 2× 字节
llama.cpp GEMV	每元素标量激活广播	标量×打包向量一条指令完成

3. CUDA Int4 指令设计参考

本章描述 CUDA 在 int4 计算上的设计方法，作为 RVV 指令设计的参考。NVIDIA 的设计选择代表了 GPU 整数计算的最先进水平。

3.1 dp4a — 四路整数点积带累积

引入: PTX ISA 5.0, CUDA 8（Pascal SM 6.1, 2016）

语义:

```
dp4a.atype.btype d, a, b, c;

// 操作:
// 四路字节点积，带 32 位累积
// d = c + (a[0] × b[0]) + (a[1] × b[1]) + (a[2] × b[2]) + (a[3] × b[3])
```

其中 `a` 和 `b` 是 32 位寄存器，各包含 4 个打包的 8 位整数，`c` 是 32 位累加器，`d` 是 32 位结果。

类型变体:

PTX 助记符	A 类型	B 类型	累加器	符号性
`dp4a.u32.u32`	u8×4	u8×4	u32	无符号×无符号
`dp4a.s32.s32`	s8×4	s8×4	s32	有符号×有符号
`dp4a.u32.s32`	u8×4	s8×4	u32	无符号×有符号
`dp4a.s32.u32`	s8×4	u8×4	s32	有符号×无符号

关键设计选择:

1. 打包寄存器格式: 4 × 8-bit 值打包到 1 × 32-bit 寄存器
2. 点积: 将 4 个乘积累加至一个 32 位累加器（非 4 个独立输出）
3. 扩展: 每个 i8×i8 乘积是 16 位; 4 个之和适合 18 位, 远在 32 位目标范围内
4. 无中间溢出: 32 位累加器可处理完整范围
5. 同族指令: `dp2a`（2 路点积用于 16 位打包值）

对 RVV 的适用性:

- 打包子字节输入产生扩展点积输出的概念可直接映射
- dp4a 使用 4×i8→i32; 我们的方案使用 4×i4→i32, 每字节打包 2 个 int4 值
- CUDA 的固定 4 路归约（匹配 32 位寄存器）与 RVV 的变长向量不同
- CUDA 使用独立类型变体助记符; RVV 可使用类似的 `.u`/`.s`/`.su` 后缀

3.2 MMA 指令 — Tensor Core Int4 支持

引入: SM 8.0（Ampere）支持 int8/int1; SM 9.0（Hopper）支持 int4

示例 PTX（Hopper）:

```
mma.sync.aligned.m16n8k128.row.col.s32.s4.s4.s32
```

语义: 矩阵乘累加, 操作于分段（fragments）:

- M=16 行, N=8 列, K=128（内维度）
- 输入 A: s4（有符号 4-bit, 每字节打包 2 个值）
- 输入 B: s4（有符号 4-bit）
- 累加/输出: s32

子字节类型的关键设计选择:

1. 分段中的子字节打包: 输入分段中每个字节保存 2 × int4 值（低/高 nibble 打包）
2. 扩展链: i4×i4 → i16（乘积）, 然后累加到 i32
3. 分段抽象: MMA 指令操作于跨 warp 线程分布的不透明"分段", 而非直接寄存器
4. 独立"稀疏"变体: `mma.m16n8k64` 使用 `.u4`/`.s4` 类型支持 2:4 结构化稀疏

与 RVV 的差异:

- CUDA MMA 分段是 warp 级抽象; RVV 指令是逐 lane 的
- CUDA K 维度固定（k128, k64）; RVV 通过 vsetvl 参数化 VL
- CUDA 打包是逐分段定义的; RVV 需要显式的字节级 nibble 排序

3.3 CUDA 方案的设计原则

原则	CUDA 实现	RVV 适配
将输入打包至更宽的容器	4×i8 → 32 位寄存器	2×i4 → 8 位字节 lane
点积带累积	dp4a 累加 4 个乘积 → 1 个累加器	vqmac 累加 4 个乘积 → 1 个累加器

原则	CUDA 实现	RVV 适配
多种符号变体	.u32, .s32, .u32.s32, .s32.u32	.vv（均有符号），.vu（无符号），.su（混合）
扩展输出防止溢出	i8×i8→i16，累加→i32	i4×i4→i8，4 个之和→i10，安全存入 i32
通过 nibble 打包子字节	MMA 分段中 s4/u4 类型	向量寄存器 lane 中每字节打包 2×i4

3.4 CUDA 未做的事情（及原因）

1. 不设通用 int4 寄存器类型：CUDA 无原生 4-bit 寄存器。始终打包至更大容器。RVV 同样打包至 8-bit SEW lanes。
2. 不设逐元素 int4 加载：子字节加载始终是打包组。CUDA 加载字节，指令内部解包。
3. 不设 int4 存储：结果始终先扩展再存储。

4. 提出的 RVV Int4 指令语义

4.1 设计哲学

提出的指令扩展现有 RVV 整数 MAC 基础设施以支持打包的 4-bit 操作数。设计遵循三个原则：

1. RVV 兼容性：与 `vsetvl`/`vsetvli`、LMUL、SEW 和掩码基础设施集成
2. 子字节打包：每字节 lane 打包 2 × int4 值，与 CUDA 的 nibble 约定一致
3. 扩展累加：int4×int4 → int8 乘积（隐式），4 个之和 → int10 部分和，累加至 int32

4.2 元素打包格式

字节内打包 int4 的布局：

```
字节位置 i:
  Bits [3:0] = 元素 2i  (低 nibble, 元素索引 0, 2, 4, ...)
  Bits [7:4] = 元素 2i+1  (高 nibble, 元素索引 1, 3, 5, ...)
```

理由：小端 nibble 字节内排序匹配：

- CUDA 的 nibble 打包约定（低 nibble = 第一个元素）
- llama.cpp Q4_0/Q4_K 权重格式（低 nibble = q4_0，高 nibble = q4_1）
- 自然的字节级内存布局

向量寄存器布局（VLEN=512，SEW=8）：

```
字节: | 0 | 1 | 2 | ... | 63 |
Nibble:| 0 | 1 | 2 | 3 | 4 | 5 | ... |126|127|
       ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
       vl=64 字节 = 128 个打包 int4 元素
```

4.3 指令 1：`vqmac.vv.i4.i32` — 向量-向量四路乘累加

用途：int4 GEMM/GEMV 的主计算指令。用单条打包操作替代解包+MAC 序列。

语义（伪代码）：

```
vqmac.vv vd, vs1, vs2, vm // 有符号×有符号变体

对每个元素组 i (0 至 vl/2 - 1):
    若掩码启用:
        // 从 2 个连续字节提取 2 个打包 int4 值
        // 每字节提供 2 个 int4 元素 (低 nibble, 高 nibble)
        // 2 字节组 → 4 个 int4 值 = 4 次乘法

        // vs1 (源 1): 字节 2i 和 2i+1
        a0 = sext4(vs1.bytes[2i] & 0x0F)
        a1 = sext4((vs1.bytes[2i] >> 4) & 0x0F)
        a2 = sext4(vs1.bytes[2i+1] & 0x0F)
        a3 = sext4((vs1.bytes[2i+1] >> 4) & 0x0F)
```

```
        // vs2（源 2）：字节 2i 和 2i+1
        b0 = sext4(vs2.bytes[2i] & 0x0F)
        b1 = sext4((vs2.bytes[2i] >> 4) & 0x0F)
        b2 = sext4(vs2.bytes[2i+1] & 0x0F)
        b3 = sext4((vs2.bytes[2i+1] >> 4) & 0x0F)

        // 点积累加
        vd.Selem_i += (int32_t)(a0*b0 + a1*b1 + a2*b2 + a3*b3)
```

寄存器配置：

- `vtype.vsew` = 32（SEW=32 用于 int32 累加/输出）
- 寄存器组：vs1 和 vs2 在字节级解释为打包 int4
- 每个 i32 输出元素消耗每输入 2 字节 = 32 bits → 8 bits → SEW/4
- EMUL 关系：若 vd 为 LMUL=m，vs1/vs2 消耗 LMUL=m/4 的寄存器空间

示例（VLEN=512，LMUL=1，vl=16）：

```
vd（int32，LMUL=1，vl=16）：16 × int32 累加元素 = 64 字节 = 512 bits（1 个寄存器）
vs1（打包 int4，LMUL=1）：16 组 × 2 字节/组 = 32 字节 → 作为打包 int4 消费
vs2（打包 int4，LMUL=1）：相同
总计：3 次寄存器读（vs1，vs2）+ 1 次寄存器读-改-写（vd）
```

LMUL 扩展（VLEN=512，LMUL=2，vl=32）：

```
vd：32 × int32 = 128 字节 = 1024 bits（2 个寄存器）
vs1：32 组 × 2 字节 = 64 字节 = 512 bits（1 个寄存器）
vs2：64 字节 = 512 bits（1 个寄存器）
```

溢出分析：

- 每乘积：i4 × i4 → i8（最小：−7×−7=49，最大：7×7=49）。适合 8 bits。
- 4 个乘积之和：±(4×49) = ±196。适合 9 bits 有符号。
- 在溢出 16 bits 前可进行 256 次累加（32768/196 ≈ 167）。
- 32-bit 累加器在溢出前可提供 >200 万次累加。

4.4 指令 2：`vqmac.vx.i4.i32` — 向量-标量四路乘累加

用途：用于 GEMV 模式——一个标量激活值乘以一个打包 int4 权重向量。这是 LLM 自回归推理中的主导模式。

语义：

```
vqmac.vx vd, rs1, vs2, vm // 有符号标量 × 有符号打包向量

// rs1 是包含 1 个 int8 激活值的字节（对标当前 GEMV）
a_scalar = sext8(rs1) // 符号扩展 int8 标量

对每个元素组 i：
    b0 = sext4(vs2.bytes[2i] & 0x0F)
    b1 = sext4((vs2.bytes[2i] >> 4) & 0x0F)
    b2 = sext4(vs2.bytes[2i+1] & 0x0F)
    b3 = sext4((vs2.bytes[2i+1] >> 4) & 0x0F)

    vd.Selem_i += (int32_t)(
        a_scalar * b0 + a_scalar * b1 + a_scalar * b2 + a_scalar * b3
    )
```

使用案例（llama.cpp GEMV）：

```
// 当前（每权重组的 3 条操作）：vand + vsrl + vwmacc.vx
// 使用 vqmac.vx（每 4 个权重的 1 条操作）：
for k in K:
    v_packed_i4 = vle8(weights + offset); // vl=16：16 字节 = 32 个打包权重
    acc = vqmac.vx.i4.i32(acc, q8_scalar, v_packed_i4);
```

```
// 2 条指令处理 32 个权重（16 个 i32 输出）
```

4.5 符号变体

遵循 CUDA 的模式和 RVV 现有约定（如 `vwmacc.vv` vs `vwmaccsu.vv`）：

助记符	vs1/vs2 类型	描述	使用场景
`vqmac.vv`	有符号×有符号	两操作数均为有符号 int4	llama.cpp Q4_K 权重 × int4 激活
`vqmacu.vv`	无符号×无符号	两操作数均为无符号 int4	ONNX RT uint4 权重 × uint4 激活
`vqmacsu.vv`	有符号×无符号	有符号 rs1 × 无符号 vs2	llama.cpp Q8_0 激活（有符号）× Q4_0 权重（无符号等价）
`vqmacus.vv`	无符号×有符号	无符号 rs1 × 有符号 vs2	ONNX RT uint8 激活 × 对称 int4 权重

4.6 与 RVV 基础设施的集成

vsetvl/vsetvli 交互:

```
// 设置 VL 用于 int32 累加输出
vsetvli t0, a0, e32, m1 // SEW=32, LMUL=1 用于输出
// VL = min(vlmax, application_N)
// 对于 vqmac.vv.i4.i32: 每个输出元素消耗每输入 2 字节
// vs1/vs2 有效元素数量 = VL × 2 字节/元素 = VL × 4 个打包 int4 值
```

LMUL 交互:

LMUL 设置	vd 元素	vs1/vs2 字节数	打包 int4 值数
m1 (vl=16)	16 × i32	32 字节	64 个打包 int4
m2 (vl=32)	32 × i32	64 字节	128 个打包 int4
m4 (vl=64)	64 × i32	128 字节	256 个打包 int4

掩码: 标准 RVV 掩码按输出元素应用。被掩码元素保留其先前的累加器值。

尾端/掩码无关: 遵循 `vtype` 中的 `vta`/`vma` 策略。

寄存器重叠约束: vd 不能与 vs1 或 vs2 的寄存器组重叠（标准 RVV 约束）。

4.7 明确不请求的内容

为了明确范围，软件团队不请求:

- 通用 int4 加载/存储: 打包内存用标准字节加载（`vle8.v`）。指令内部解包。无需 `vle4.v`。
- Int4 SEW: 基础 SEW 保持 8 或以上。不改变 RVV 的元素宽度基础设施。Int4 打包是指令内部的。
- Int4→float 转换: 去量化仍是在 int32 累积后使用现有 `vfcvt.f.x.v` 的独立步骤。
- Int4 置换/混洗: 字节 lane 上的标准向量操作已提供足够灵活性。
- 逐 nibble 掩码: 掩码是逐输出元素组（4 个乘积）的，而非逐单个 int4 值。

5. 软件需求汇总

5.1 所需指令（按负载影响力排序）

#	指令	负载	影响
1	vqmac.vx.i4.i32（有符号标量 × 打包 int4 → int32）	llama.cpp GEMV（53.93%）	消除解包 + 2 操作 MAC → 单操作
2	vqmacu.vv.i4.i32（无符号×无符号，均打包）	ONNX RT QGEMM（74.51%）	替代 vwmulu+vwaddu 操作对
3	vqmacsu.vv.i4.i32（混合有/无符号）	两负载的混合精度场景	支持混合 int4/int8 格式

5.2 不可妥协的需求

1. 扩展累加至 int32：4-bit 乘积在内维度跨多个累加后会迅速超 int16 范围（K > 167）。
2. 小端 nibble 打包：低 nibble = 第一个元素，高 nibble = 第二个。匹配现有权重格式。
3. SEW=32 输出：累加/输出以 SEW=32（int32）操作。输入打包是隐式的（每输出元素 2 字节）。
4. 掩码支持：标准 RVV 按输出元素掩码用于部分向量处理。
5. vsetvl 兼容：VL 以输出元素（int32 数量）指定，与现有 RVV 整数 MAC 指令相同。

5.3 锦上添花的需求

1. vqmac.vv.i4.i16 变体：用于 int16 累加即足够的场景（较小 K 维度），减少寄存器压力。
2. 8 个打包 int4 的扩展 vqmac：最大密度（8 × int4 × int4 乘积 → int32），虽然超出典型 K 组规模。
3. 尾端处理提示：可选标志指示尾端元素为零，避免独立的尾端处理。

5.4 量化方案兼容性

所提出的指令必须支持任务 1 和任务 2 中验证过的量化方案：

方案	符号性	输入范围	vqmac 变体
对称 int4（llama.cpp）	有符号×有符号	[-7, 7]	vqmac.vv / vqmac.vx
非对称 uint4（ONNX RT）	无符号×无符号	[0, 15]	vqmacu.vv / vqmacu.vx
混合 int8 激活 × int4 权重	有符号×无符号	[-127,127] × [0,15]	vqmacsu.vv
Q4_0（对称，无零点）	有符号	[-7, 7]	vqmac.vv
Q4_K（缩放+对称）	有符号	[-7, 7]，逐块缩放	vqmac.vv + 现有缩放操作

5.5 预期性能影响

假设 VLEN=512 和现有内核结构。实际硬件收益取决于微架构。

负载	内核	当前内循环	使用 vqmac	预估加速比
llama.cpp GEMV	Q4_0×Q8_0	vand + vsrl + vwmacc（3 操作/8 权重）	vqmac.vx（1 操作/32 权重）	~2.5×
llama.cpp GEMM	Q4_K×Q8_K	vand + vsrl + vwmacc + vwmacc（缩放）	vqmac.vv + vwmacc（缩放）	~2.0×
ONNX RT QGEMM	u8×u8→i32	vle8 + vwmulu + vwaddu（3 操作/K）	vle8 + vqmacu（2 操作/K）	~3.4×

注意：这些是内循环算术加速比。端到端加速取决于这些内核的运行时占比（llama.cpp 53.93%，ONNX RT 74.51%）。

6. 与 RISC-V Matrix Extension (IME) 的关系

RISC-V Integrated Matrix Extension（IME，Option G）是 RISC-V 社区定义使用向量寄存器的矩阵乘累加指令的持续努力。我们的方案是对 IME 的补充：

方面	我们的方案（vqmac）	IME/Option G
范围	向量指令（1D）	矩阵指令（2D tile）
输入类型	向量寄存器中打包 int4	带 s4/u4 类型的矩阵分段
输出	int32 向量	int32 矩阵 tile
K 维度	每输出元素 4 路点积	可配置（k128，k64 等）
适用场景	中等 K 的 GEMM、GEMV	固定 tile 大小的大型 GEMM
集成方式	扩展现有 RVV ISA	RVV 旁的新扩展

`vqmac` 指令与 IME 是互补的，而非竞争关系。对于低批量 LLM 推理（batch=1 自回归），面向 GEMV 的 vqmac 是合适的基础原语。对于高批量训练或提示处理，IME 式的矩阵指令效率更高。

7. 参考文献

内部

1. llama.cpp Int4 精度报告：`docs/report/int4-instruction/llama-int4-precision-2026-04-28.md`
2. ONNX RT YOLO Int4 精度报告：`docs/report/int4-instruction/yolo-int4-precision-2026-04-28.md`
3. llama.cpp Q4_K GEMM 内核：`applications/llama.cpp/rvv-patches/gemm-q4_K-8x4-q8_K/rvv_gemm_q4_K_8x4.inl`
4. ONNX RT QGEMM 内核：`applications/onnxrt/rvv-patches/qgemm-kernel-vl16/rvv_qgemm_kernel_vl16.inl`
5. Int4 指令设计计划：`docs/plans/int4-instruction-design-2026-04-28.md`

外部

6. NVIDIA PTX ISA — dp4a 指令：CUDA Parallel Thread Execution ISA，第 9.7.1.23 节
7. NVIDIA PTX ISA — MMA 指令：CUDA Parallel Thread Execution ISA，第 9.7.14 节（Tensor Core Operations）
8. RISC-V Vector Extension 1.0：`riscv/riscv-v-spec`
9. RISC-V Integrated Matrix Extension (IME)：`riscv.atlassian.net/wiki/spaces/IMEX`
10. CUDA C Programming Guide — Warp Matrix Functions：附录 K

附录 A：指令编码占位符

软件团队不提出指令编码。此占位符指示硬件团队需定义的位置：

```
vqmac.vv.i4.i32 vd, vs1, vs2, vm
vqmac.vx.i4.i32 vd, rs1, vs2, vm
vqmacu.vv.i4.i32 vd, vs1, vs2, vm
vqmacsu.vv.i4.i32 vd, vs1, vs2, vm

[ENCODING: 硬件团队定义 funct6/funct3/opcode 字段]
[CSR 依赖：无，超出已有 vtype/vl]
[异常行为：与现有 RVV 整数 MAC 指令相同]
```

附录 B：验证测试向量

软件团队将提供：

1. 来自两种负载的打包 int4 权重矩阵（真实模型数据）

2. 预期 int32 累积结果用于正确性验证
3. 边界情况: 零值、最大范围值（–7, 7, 0, 15）、混合符号
4. LMUL 扩展测试用例（m1, m2, m4）
5. 掩码交互测试用例（部分 VL, 被掩码元素）

附录 C: 术语表

术语	定义
GEMV	通用矩阵-向量乘法（1 行 × 矩阵）
GEMM	通用矩阵-矩阵乘法
QGEMM	量化 GEMM（uint8×uint8→int32）
Q4_0	llama.cpp 4-bit 块量化（对称、无零点、逐块缩放）
Q8_0	llama.cpp 8-bit 块量化（对称、逐行缩放）
MAC	乘累加（Multiply-Accumulate）
SEW	标准元素宽度（RVV 概念）
LMUL	向量寄存器组乘数
VL	向量长度（处理的元素数）
CosSim	余弦相似度（与 FP32 基线比的输出质量指标）
dp4a	CUDA 指令: 4 路字节点积带 32 位累积
vqmac	提出的 RVV 指令: 带打包 int4 的向量四路乘累加
nibble	4-bit 值（半字节）